08: Memory Protection

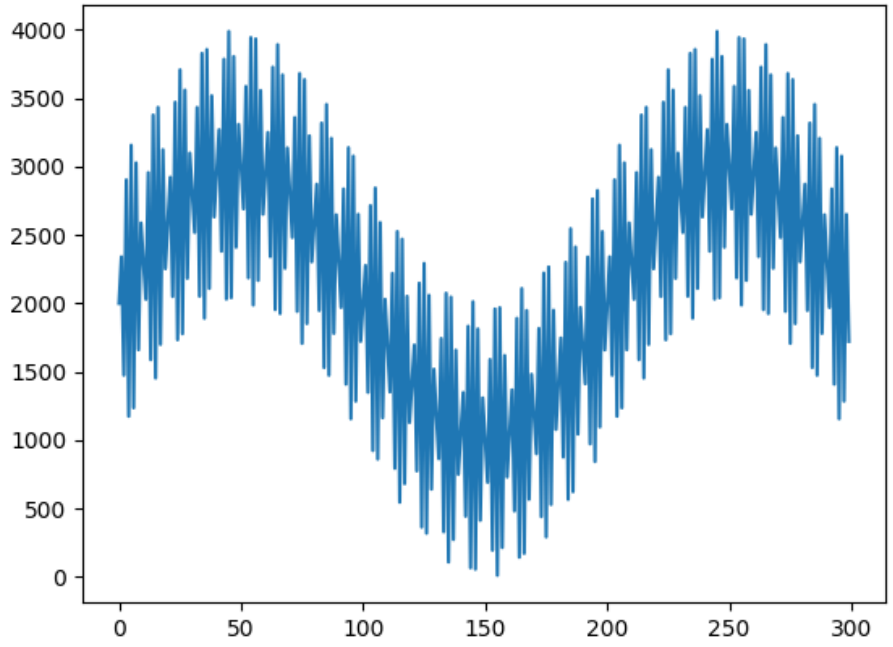
Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

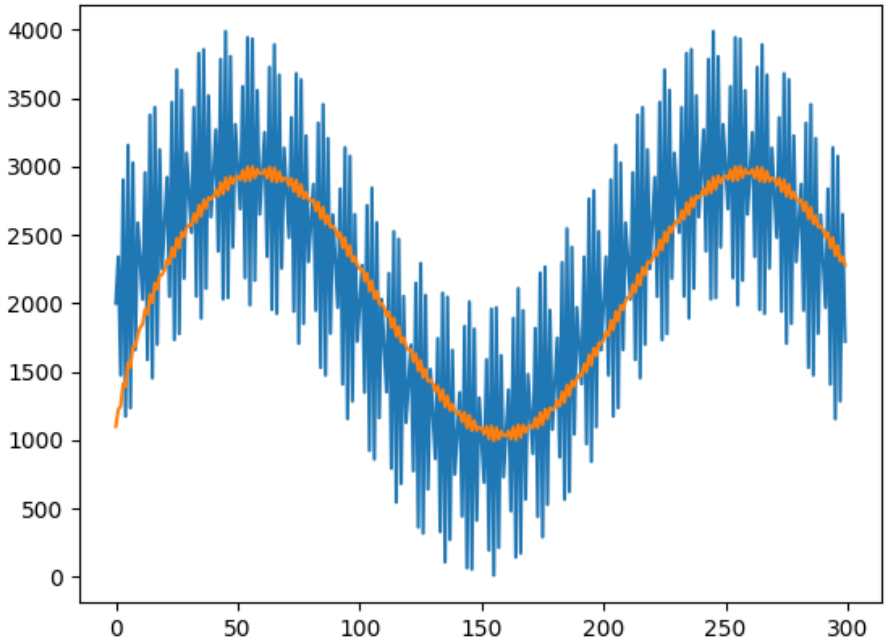
Let’s talk P3

- New Version: V5
- ILA tutorial finalized

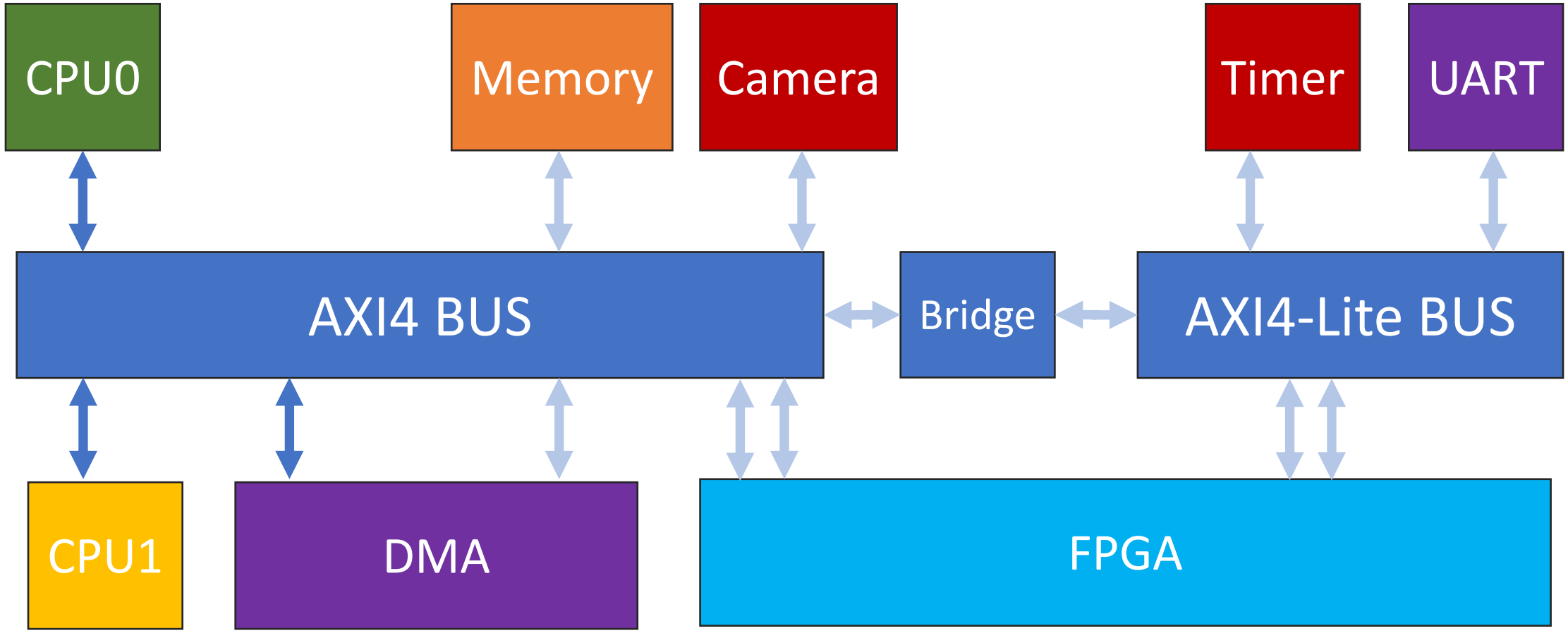
P3: Filtering with DMA



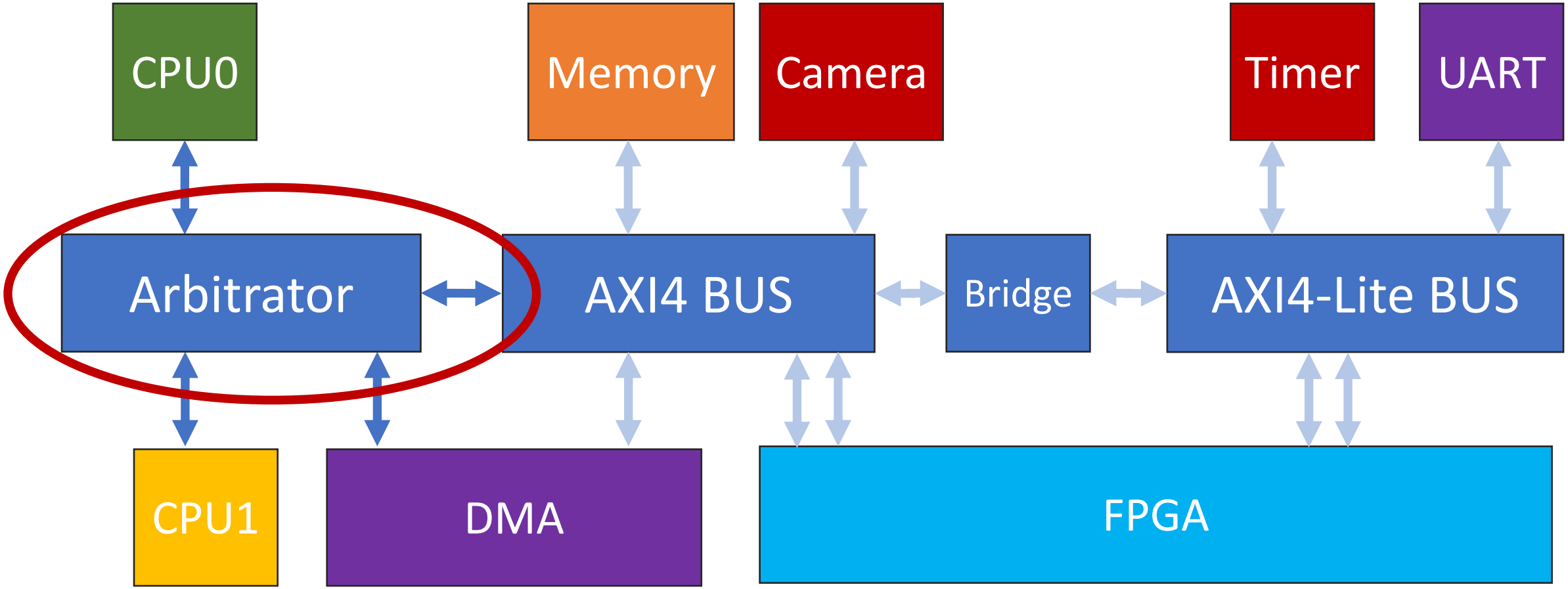
->



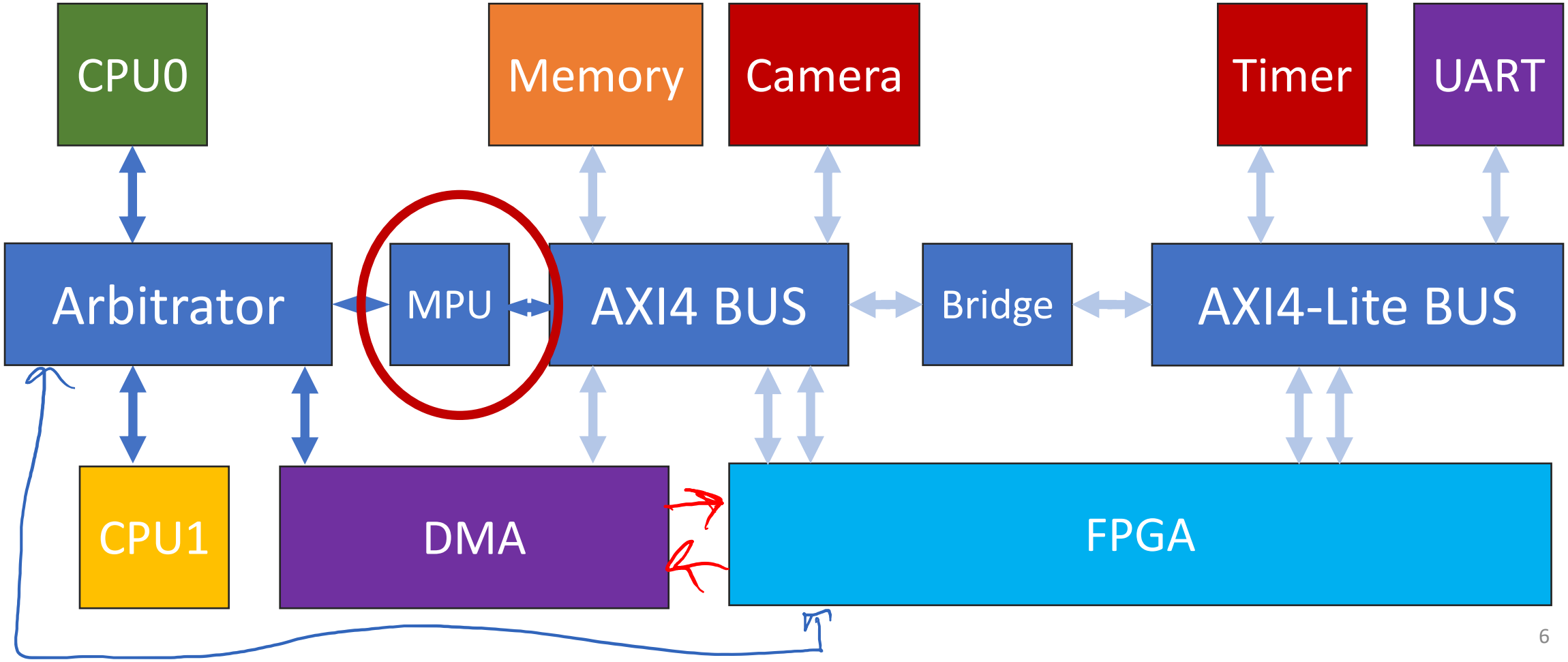
Machine Model, V1



Machine Model, V2

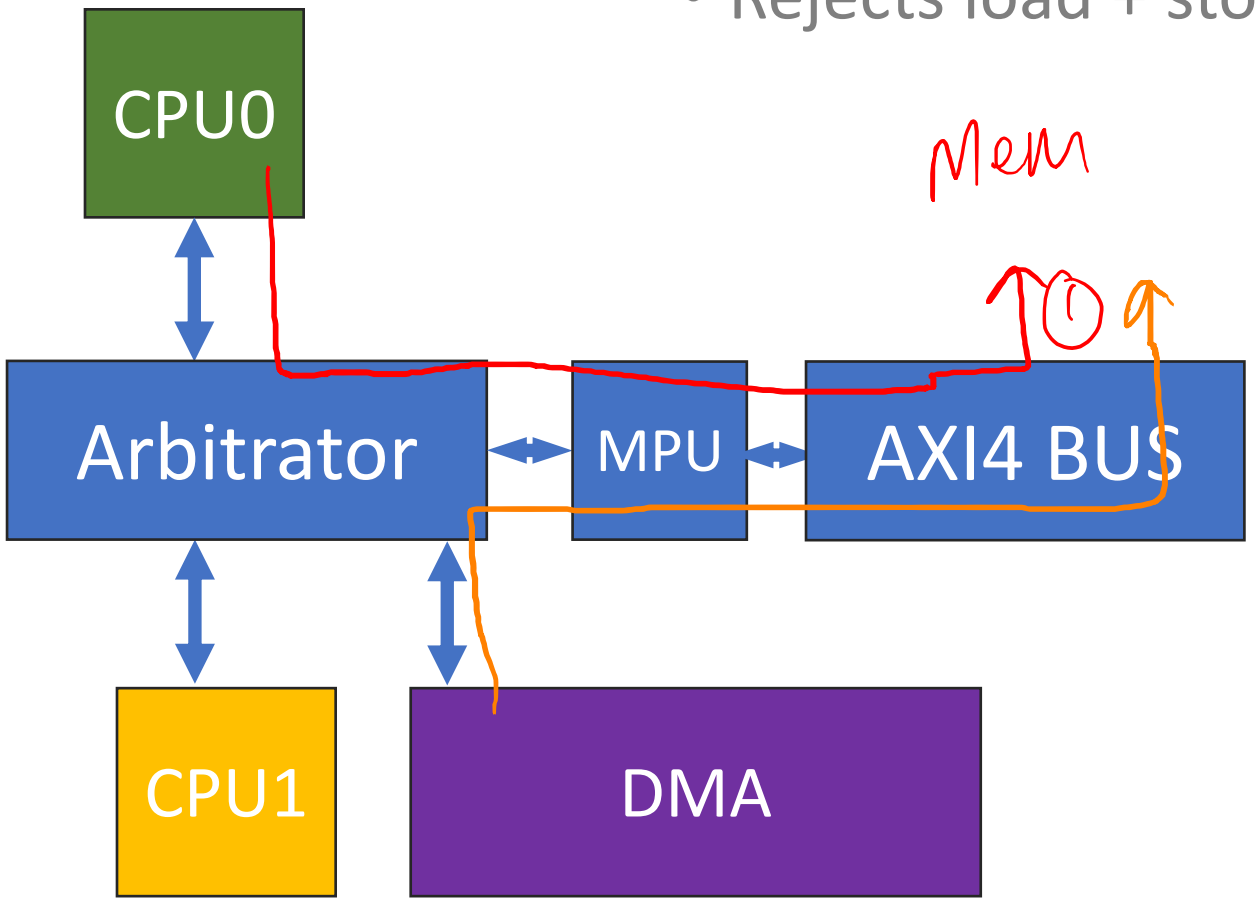


Machine Model, V3



MPU: Memory Protection Unit

- Rejects load + stores that are "unauthorized"



1 CPU loads from mem

=> mpu => Authorized!

2 DMA loads from mem

= mpu => Not Authorized!

Why do you care?

- This shows up in P3.
- And it's hard to debug.

* great for security

MPU Table

From CPU1 DMA

CPU load 0x1234

CPU store 0x1234

Addr	Source	Permission	
0x1234	CPU0	load	!store
0x1234	DMA	!load	!store
0xabcd	CPU0	!load	store

to Mem or FPGA "Bus"

MPU Table

Covers 0x1200 – 0x12ff

OS Mode

BASE Addr	Offset	Source	Permission
0x1200	0xff	CPU0	Read, !write
"Code"	sizeof(code)	CPU	Read, !write
"data"	sizeof(data)	CPU	Read, Write
"data"	"	DMA	Read, !write
"DMAdata"		CPU	R, W
"DMAdata"	"	DMA	R, W

MMIO

MPU Steps

1. CPU Writes Table
2. CPU enables MPU
3. CPU/DMA uses MPU

OS mode

4. CPU disables MPU
5. CPU updates MPU
6. CPU re-enables MPU

OS mode

Tangent: MPU vs. MMU

- MPU: Memory Protection Unit
- MMU: Memory Management Unit

- MPU: prevents unauthorized reads/writes

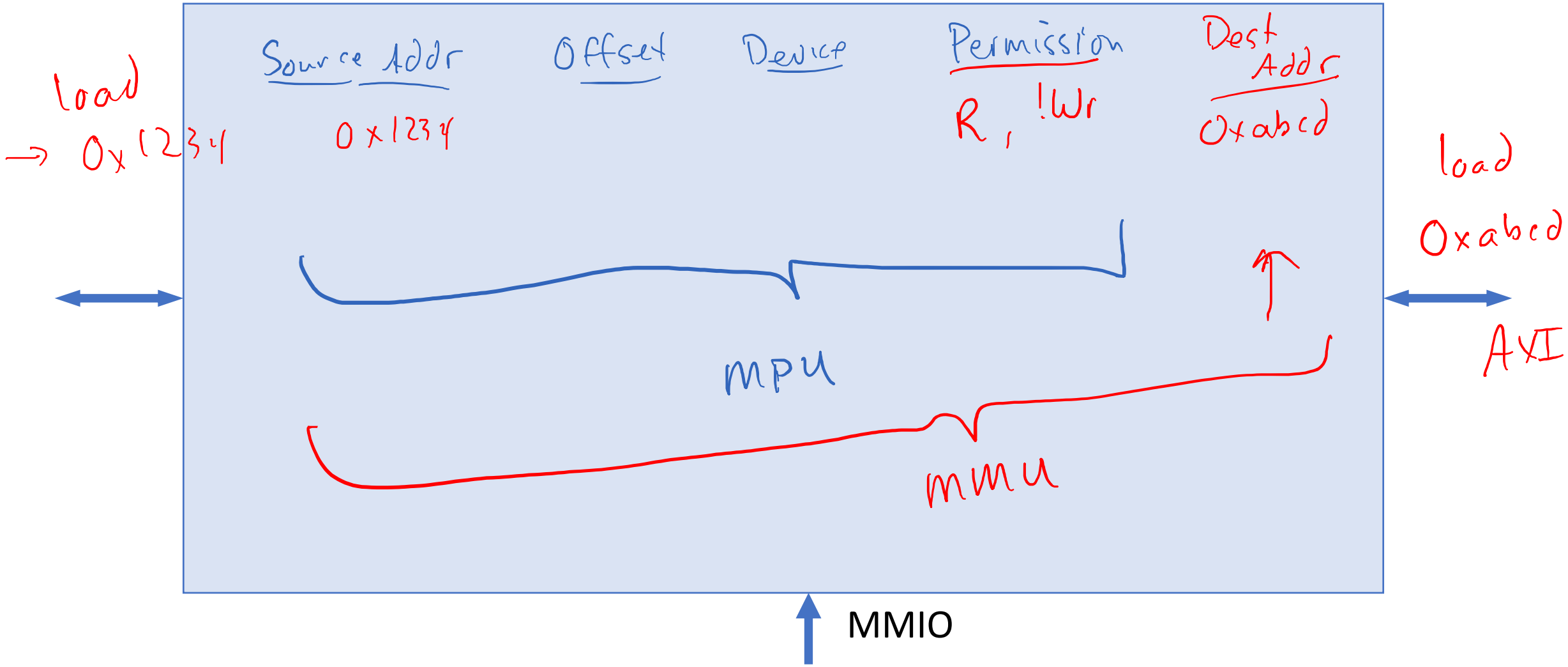
- MMU: prevents unauthorized reads/write

--- and ---

re-address reads/writes

load 0x1234 → ~~0xabcd~~ 0x456f

MMU vs. MPU Table



Back on Topic: Memory Protection in P3

```
uint32_t PING [MAX_PKT_LEN];
uint32_t PONG [MAX_PKT_LEN];

...
```

XAxiDma_TxRxBuffer(… , PING, PONG, **sizeof**(PING));

Why doesn’t this work?

```
uint32_t PING [MAX_PKT_LEN];
uint32_t PONG [MAX_PKT_LEN];

...
```

XAxiDma_TxRxBuffer(… , PING, PONG, **sizeof**(PING));

What can we do about it?

```
uint32_t PING [MAX_PKT_LEN];
uint32_t PONG [MAX_PKT_LEN];

...
```

XAxiDma_TxRxBuffer(… , PING, PONG, **sizeof**(PING));

Memory Sections:

.text → code instructions, R, !write

.rodata Read-only data R, !WR

.data → most of appln data, R, Wr

.heap → malloc/free data R, Wr

.stack → local vars, funct returns R, Wr

Memory Sections in MPU:

.text

.rodata

.data

.heap

.stack

Let's add a new "Section"

- .dma_data

- MPU:

Addr	Device	Permissions
dma_data	DMA	R, Wr
"	CPU	R, Wr

Telling GCC to use ”.dma_data”

```
__attribute__ ((section(".dma_data")));
```

- GCC specific, may not work on other compilers!

Using ”.dma_data” works!

```
uint32_t PING [MAX_PKT_LEN] __attribute__ ((section(".dma_data")));
uint32_t PONG [MAX_PKT_LEN] __attribute__ ((section(".dma_data")));

...

XAxiDma_TxRxBuffer( … ,  PING, PONG, sizeof(PING));
```

Rules

- Default: don’t use “.dma_data”
- DMA buffers: always “.dma_data”
- DMA read-only buffers: either way
- DMA write-only buffers: always “.dma_data”

Next Time:

- Performance Analysis

08: Memory Protection

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University